Adaptive Network Traffic Anomaly Classification

Track 1 Final Capstone • Educational Prototype

Adaptive Network Traffic Anomaly Classification

Track 1 Final Capstone — Problem 01

Domain: Cybersecurity | **ML Task:** Binary Classification | **Prototype:** Educational

Abstract

This project implements an end-to-end machine-learning workflow covering data inspection, EDA, preprocessing, model training, evaluation, interpretation and a Streamlit demonstration. The design follows the supplied capstone brief: free/open-source tools, a foundational classifier, measurable evaluation and a lightweight prototype.

1. Problem Definition

Problem statement: Classify network-flow records as normal or anomalous using measurable traffic characteristics.

Objective: Develop an end-to-end, zero-cost ML solution that investigates the data, prepares it, builds and evaluates a foundational model, interprets results and demonstrates prediction through a lightweight application.

Expected inputs: flow duration; packet count; byte count; packet rate; protocol/flow attributes

Expected output: normal or anomalous with optional risk indication

2. Research Questions

1. Which measurable features provide useful signal?
2. What preprocessing is necessary?
3. How does a foundational classifier perform?
4. What errors require attention?

3. Dataset

The packaged demo dataset is synthetic and reproducible with seed 42 because the supplied brief specifies feature families and deliverables but does not include a raw assigned dataset. For a formal submission using an instructor-specified/public dataset, replace the CSV and add its original citation.

4. EDA and Data Quality

The notebook checks missing values, duplicates, class balance, distributions, relationships, outliers and potential leakage. These checks are kept explicit for reproducibility and viva defense.

5. Methodology

1. Load data and separate target.
2. Stratified train/test split.
3. Impute missing values where required.

4. Scale numerical features.

5. Encode categorical variables when required.

6. Train Logistic Regression with balanced class weights.

7. Evaluate on held-out data.

8. Save the complete preprocessing + model pipeline.

9. Reuse the same pipeline in Streamlit.

6. Model Choice

Logistic Regression is a fast, interpretable foundational method and is explicitly within the capstone's expected model family. Deep learning is intentionally outside the default scope.

7. Results

Metric	Value
Accuracy	0.8688
Precision	0.4124
Recall	0.8696
F1-score	0.5594
ROC-AUC	0.9317

Confusion matrix: `[[377, 57], [6, 40]]`

These scores are hold-out results on synthetic data. They demonstrate the implementation and reproducibility, not real-world cybersecurity effectiveness.

8. Prototype

The Streamlit app validates user input, creates a feature row with the training schema, invokes the persisted pipeline and displays a prediction with probability/risk.

9. Error Analysis

False positives can increase unnecessary review; false negatives can let risky observations pass. Threshold selection should therefore be driven by the intended operational cost when real data are available.

10. Limitations

Synthetic data are not representative of all real networks; traffic drift, noisy labels and unseen attack patterns are not modeled.

11. Future Scope

- Replace synthetic data with a documented assigned/public dataset.

- Compare Logistic Regression, Decision Tree and KNN.

- Add cross-validation, threshold analysis and calibration.

- Add explainability and monitoring.

- Add automated tests and CI.

12. Reproducibility

Seed 42, packaged CSV, source code, notebook, trained pipeline, requirements and README are included.

13. Viva Questions

1. Why Logistic Regression?
2. Why stratified splitting?
3. Why must inference preprocessing match training preprocessing?
4. What does recall mean here?
5. What is data leakage?
6. What is a false negative cost?
7. Why are synthetic metrics not production evidence?
8. How would you validate on a real dataset?

References

- Learn Depth Academy LLP, Track 1 Final Capstone, Problem 01.
- scikit-learn documentation (modeling and metrics).
- Streamlit documentation (application prototyping).